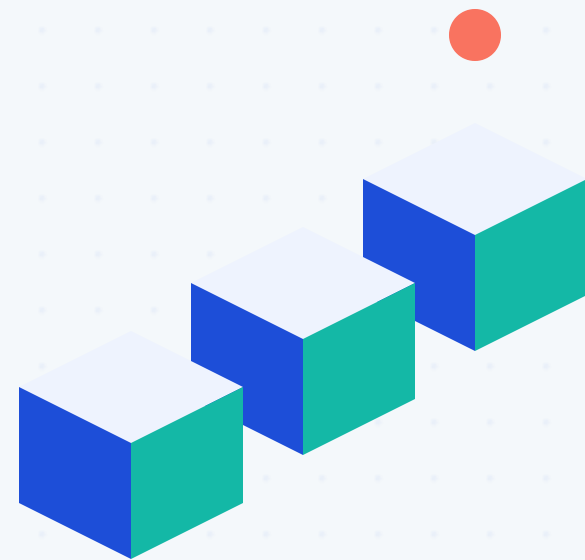


WEEK 03 · 지휘하고 판정하고 공개하기

에이전트 팀과 함께, 오늘 세상에 내놓습니다



> _ 담당을 나눠서 검토하고, 내가 판정할게

3주차

오늘의 약속

지난주까지

AI가 작성하고

사람이 검토하고

+

오늘 붙는 두 칸

사람이 공개를 결정하며

공개 책임도 사람이 집니다

공개할지 정하는 것도, 공개한 뒤 책임지는 것도 사람입니다.

오늘부터 "AI"는 한 명이 아닙니다

지금까지

대화창 하나

AI 하나와 대화했습니다.

오늘

HTML

CSS

JS

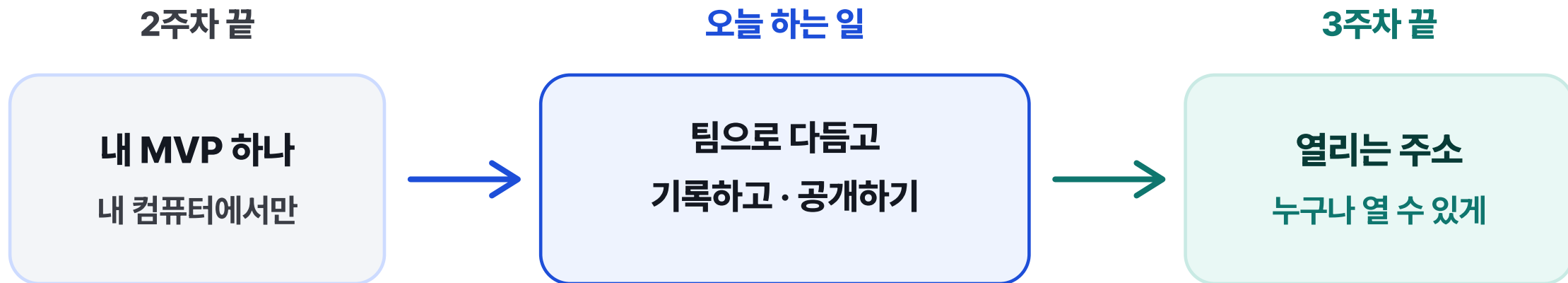
QA

네 담당이 각자 다른 파일을 맡습니다.

담당이 다르면 **의견도 갈립니다**. 그건 고장이 아니라 정상입니다 — 갈렸을 때 누구 말을 들을지 정하는 사람, 그게 오늘 여러분의 자리입니다.

지난주에서 이번 주로

2주차에는 내 MVP를 하나 완성했습니다. 3주차에는 그걸 팀으로 다듬고 · 기록하고 · 공개합니다.



쓰는 폴더는 하나입니다 - 2주차에 만든 그 폴더

새 프로젝트를 만들지 않습니다. 진행하던 폴더 그대로 씁니다.

오늘 배우게 될 것

여섯 정거장, **실습 15개**입니다
정거장마다 손에 남는 결과가 하나씩 생깁니다

1

시작 상태 보호와 하네스

실행 확인 · 백업 · 공개 금지 파일

2

싱글 · 서브 · 멀티 에이전트

싱글 · 서브 · 멀티

3

담당 나누기와 팀 규칙

HTML · CSS · JS · QA

4

병렬 검토 · 결과 판정 · 구현

엇갈린 결과를 근거로 정합니다

5

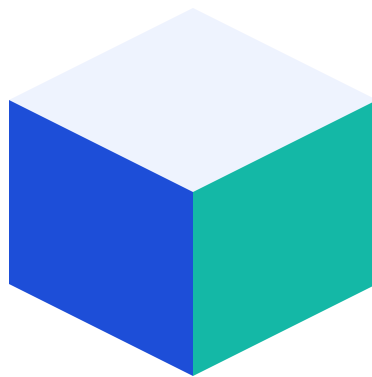
플러그인 · MCP · GitHub 기록

저장소 · 커밋 · 안전 점검

6

공개와 공유 범위

공유 범위 · URL 자가 점검



PART 1 / 6

시작 상태 보호와 하네스

손대기 전에, 지금을 지킵니다.

PART 1은 이렇게 배웁니다

백업이 끝난 다음에야 코드를 만집니다. 순서 자체가 오늘의 교훈입니다.



지난주에 배운 용어

하네스

맡기기 전에 **목표 · 읽을 자료 · 권한 · 완료 조건**을 정해 두고, 끝나면 확인하는 작업 틀입니다.

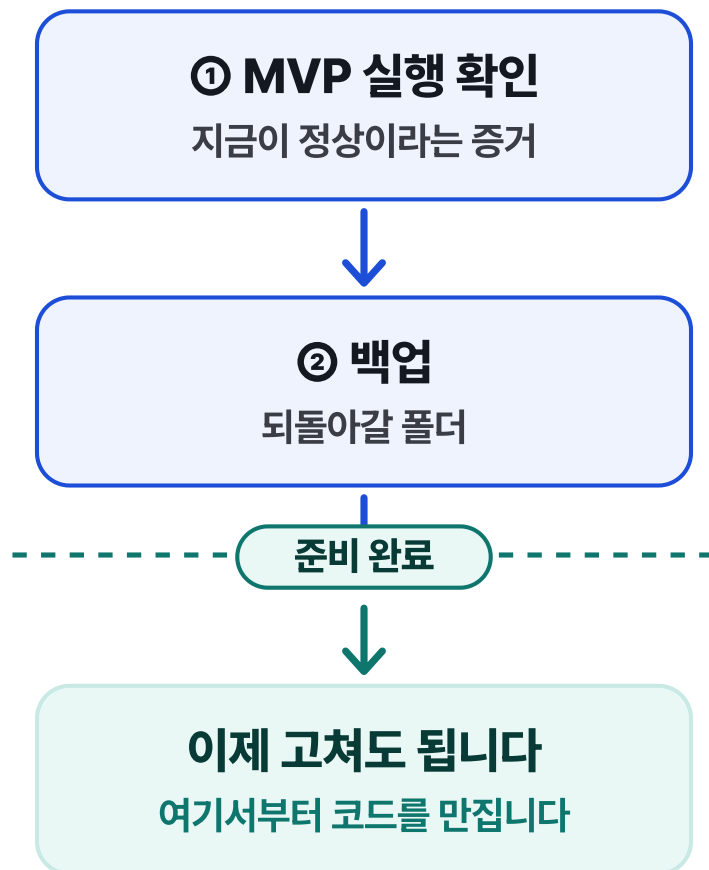
모두가 합의한 표준 용어는 아니고 **이 수업에서 쓰는 이름**입니다.

등산할 때 몸에 매는 안전벨트와 같습니다 — 떨어지는 걸 막는 게 아니라, **떨어져도 되돌아올 수 있게** 미리 묶어 두는 것.

- 1 **목표**
무엇을 끝내려는가
- 2 **읽을 자료**
무엇을 보고 판단하는가
- 3 **권한**
어디까지 손대도 되는가
- 4 **완료 조건**
무엇을 보면 끝인가
- 5 **사후 확인**
끝난 뒤 무엇을 검사하는가

고치기 전에 해 둘 두 가지

코드에 손대기 전에 할 일은 딱 둘입니다. 이 둘이 끝나야 시작합니다.



1 MVP 실행 확인
고장 난 채로 시작하면 무엇 때문에 깨졌는지 알 수 없습니다.

2 백업
잘못 고쳤을 때 돌아갈 자리가 없습니다.

"파일이 있다"와 "작동한다"는 다릅니다

파일 목록으로 보면

- index.html 있음
- style.css 있음
- script.js 있음
- 전부 그대로 있으니 괜찮아 보입니다.

≠

실제로 열어 보면

- 브라우저가 그새 업데이트됐을 수 있습니다.
- 마지막에 고치다 만 부분이 남았을 수 있습니다.
- 목록으로는 알 수 없습니다.

나중에 발견하면 **원인을 찾기 어렵습니다.** 그래서 지금 확인하는 게 가장 좋습니다.

실습 ①

MVP 실행 확인

완성 기준 지금이 "정상" 이라는 증거가 손에 있음

- 1 index.html을 열고 대표 흐름을 끝까지 한 번 실행합니다.
- 2 중간에 멈추지 말고 결과 화면이 나올 때까지 갑니다.
- 3 정상 화면을 캡처하거나 메모로 남깁니다 — 오늘 뭔가 깨지면 이게 비교 기준이 됩니다.

... 이렇게 말하세요

```
>_ 지금 이 프로젝트가 실제로 작동하는지, index.html을 열어서 대표 흐름을 처음부터 끝까지 확인해 줘.  
고치지는 말고 결과만 알려 줘.
```

용어

백업

지금 상태를 **그대로 복제해 따로 두는 것**입니다.

오늘은 여러 담당이 서로 다른 파일을 건드립니다. 한 곳이 잘못돼도 **통째로 되돌아올 수 있어야** 합니다.

게임에서 보스 앞에 세이브하는 것, 문서를 고치기 전에 「다른 이름으로 저장」해 두는 것과 같습니다.

내 프로젝트

지금 상태 — 원본

↳ 복제

백업_2026-08-08

복제본 — 손대지 않습니다

↳ 원본에서 계속

내 프로젝트

여기서 계속 작업합니다

실습 ②

백업 만들기

완성 기준 백업본과 작업본이 이름만 봐도 구분됨

- 1 프로젝트 폴더를 통째로 복제합니다.
- 2 폴더 이름에 백업과 오늘 날짜를 넣습니다 — 어느 쪽이 원본인지 헷갈리지 않게.
- 3 복제한 뒤 작업은 원래 폴더에서 계속합니다.

... 이렇게 말하세요

```
>_ 지금 폴더 상태를 그대로 복제해서 백업_2026-08-07 폴더로 남겨 줘. 원본은 그대로 두고 작업은 지금  
폴더에서 계속할게.
```

이제 처음으로 코드를 만집니다

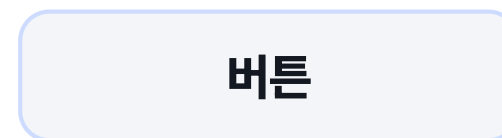
백업이 끝났으니 이제 **안전하게** 고칠 수 있습니다.

첫 수정은 가볍게 — 같은 화면도 작은 움직임 하나로 인상이 달라집니다.

마우스 오버 효과

마우스를 올렸을 때만 잠깐 반응하는 변화입니다. 기능을 바꾸는 게 아니라 **살아 있는 느낌**을 더하는 일입니다.

① 평소



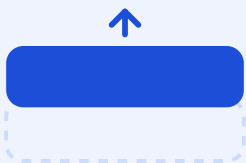
마우스를 올리면

② 살짝 떠오릅니다



같은 버튼입니다. 마우스를 올렸을 때만 움직입니다.

참고할 움직임 세 가지



버튼

마우스를 올리면 살짝 떠오릅니다.



카드

스크롤하면 부드럽게 나타납니다.



아이콘

누르면 눌린 만큼 반응합니다.

따라 만드는 게 아니라 **느낌만** 가져옵니다. 2주차의 「복사가 아니라 관찰」과 같습니다.

실습 ③

내 화면에 움직임 하나 넣기

완성 기준 마우스를 올리면 움직임이 보이고, 나머지는 그대로 임

1 세 가지 중 하나만 고릅니다.

2 내 페이지의 버튼이나 카드에 그 효과를 넣습니다.

3 다른 기능과 배치는 그대로 둡니다.

... 이렇게 말하세요

> 내 페이지에서 버튼(또는 카드)에 마우스를 올리면 살짝 떠오르는 효과를 넣어 줘. 0.2초 정도로 과하지 않게. 다른 기능과 배치는 그대로 뒀어.

AGENTS.md는 조언이지 자물쇠가 아닙니다

AGENTS.md는 에이전트가 먼저 읽는 안내문입니다.

규칙이 해 주는 것

무엇을 먼저 읽을지 알려 줍니다.

어떤 기준으로 판단할지 알려 줍니다.

방향을 줍니다.

규칙이 못 하는 것

여기저 못하게 막지는 못합니다.

그래서 권한과 확인이 늘 함께 붙습니다.

결과를 보증하는 건 확인입니다.

도서관 「정속」 안내문과 같습니다 — 방향은 주지만, 조용하게 만드는 건 결국 사서입니다.

밖에 나가면 안 되는 파일부터 걸러 둡니다

오늘 이 프로젝트는 커밋되고, 공개됩니다. 지금 미리 확인합니다.

비밀번호

파일 안에 적어 둔 적 없나요?

API 키

연습하며 붙여 넣지 않았나요?

개인 연락처

전화번호·이메일이 예시로 남았나요?

남의 자료

가져다 쓴 이미지나 글이 있나요?

내 컴퓨터 경로

내 사용자 이름이 든 경로가 남았나요?

있으면 지금

빼거나 **가짜 값으로 바꿉니다.**

한 번 나간 것은 지워도 흔적이 남습니다. **초반에 안전 기준을 잘 정해 두어야** 나중에 문제를 줄일 수 있습니다.

이제 담당을 나눌 준비가 됐습니다



실행 확인



백업



수정



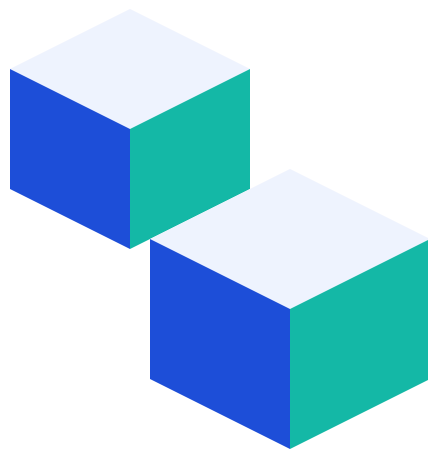
공개 금지 파일

다음 질문은 "몇 개를 띄울까"가 아닙니다.

"어떤 일이 서로 독립적인가" — 나눌 수 있는 일이 있어야 나누는 의미가 생깁니다.

PART 2

→ 판단 기준부터 잡습니다



PART 2 / 6

싱글 · 서브 · 멀티 에이전트

AI를 쓰는 방식은 하나가 아닙니다.

PART 2는 이렇게 배웁니다

이번 파트는 개념만 합니다. 실습은 PART 3부터입니다.



왜 나눠서 시킬까요

회사도 기획 · 디자인 · 개발 · 검수를 한 사람이 다 하지 않습니다.

한 명이 처음부터 끝까지



느리고, 놓치는 게 생깁니다.

특히 자기가 방금 만든 것은 자기 눈에 안 보입니다.

담당을 나누면



각자 자기 구역만 봅니다.

보는 범위가 좁아지면 더 깊이 보입니다.

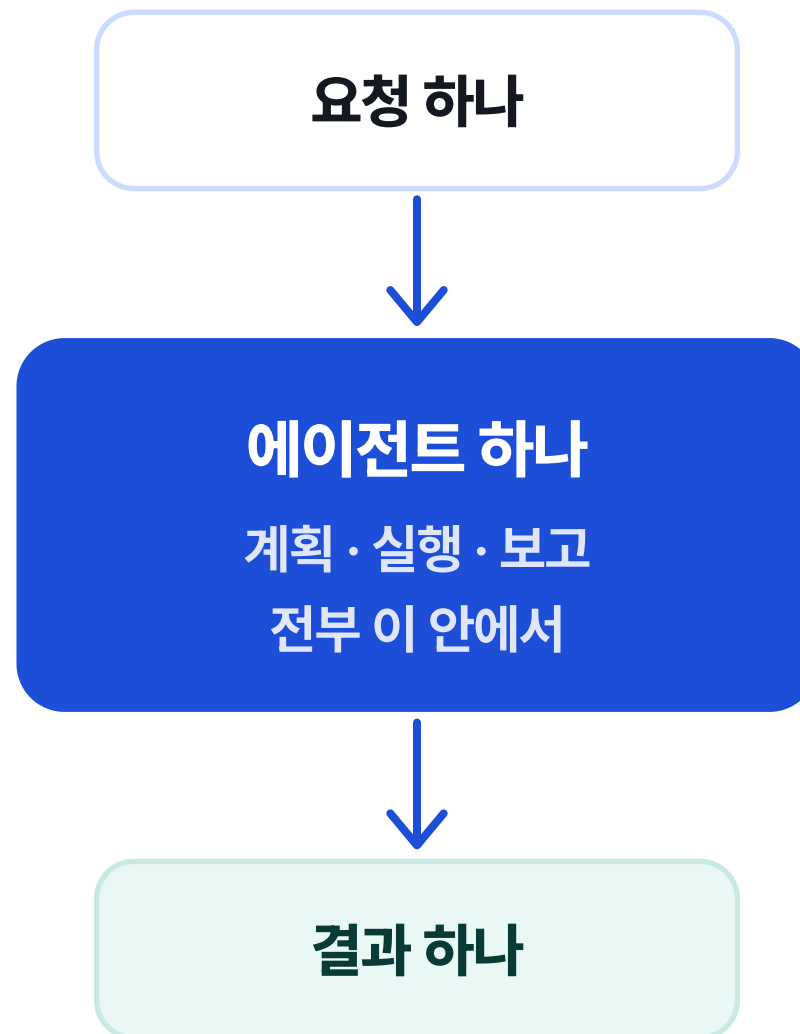
용어

싱글 에이전트

하나의 에이전트가 계획 · 실행 · 보고를 혼자 다 하는 방식

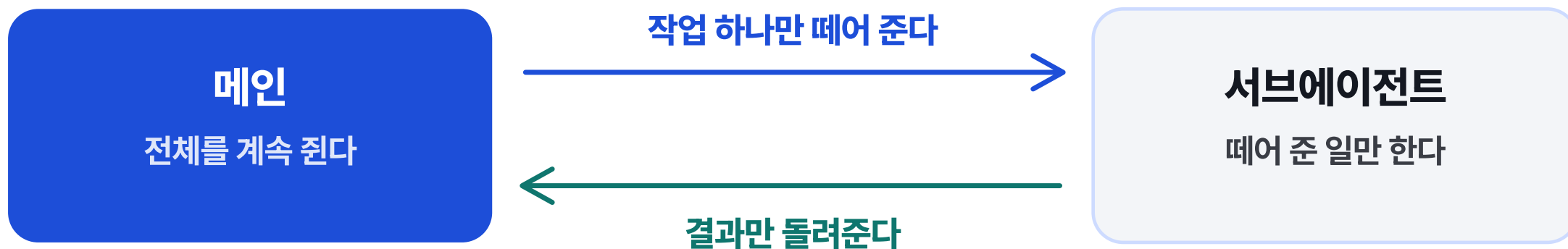
지금까지 여러분이 써 온 방식입니다. 새로 배우는 게 아니라 이름을 붙이는 겁니다.

한 번 요청하면 한 결과가 돌아옵니다. 작은 수정 하나에는 이게 가장 빠릅니다.



서브에이전트

메인이 특정 작업 하나를 떼어 다른 에이전트에게 맡기는 것 — 통제권은 **메인이 계속 쥐고 있습니다.**



요리하다가 "마늘만 다져 줘"라고 부탁하는 것과 같습니다 — 요리는 여전히 내가 합니다.

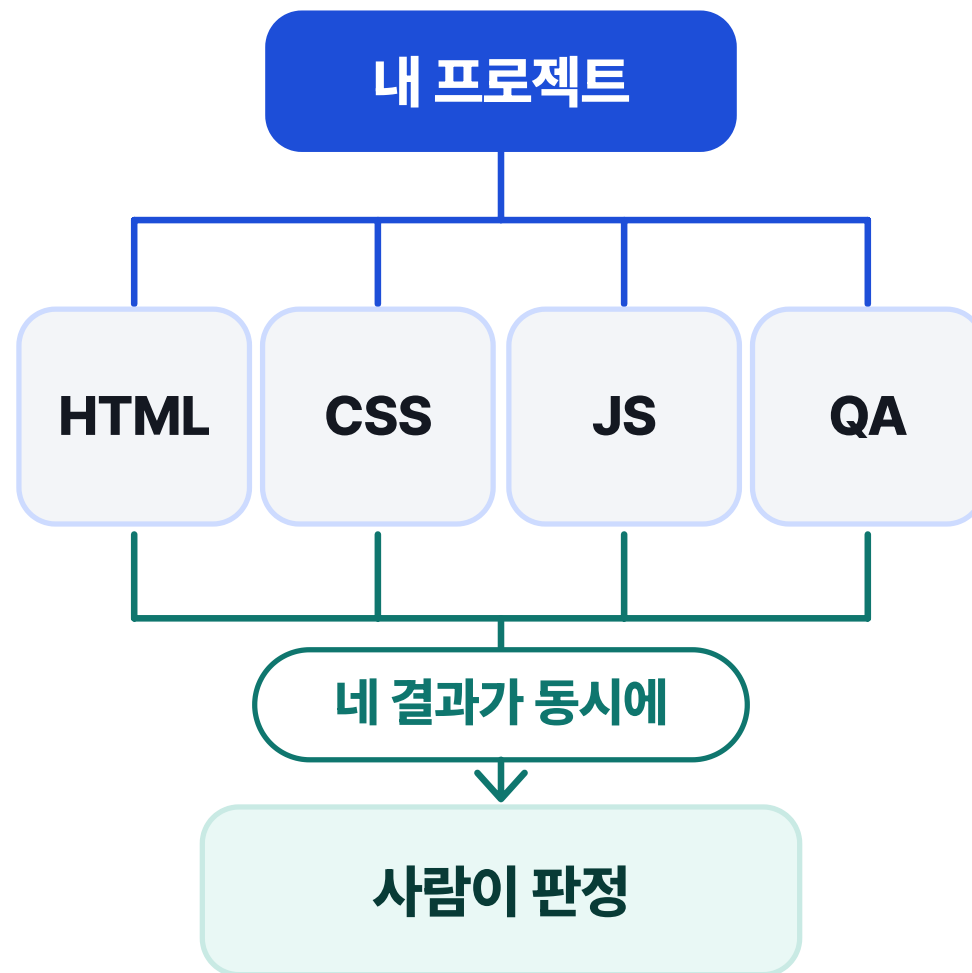
용어

멀티에이전트

하나의 프로젝트를 여러 전문 역할로 나눠, 각자 담당 구역을 살펴보게 하는 방식

여러 결과가 **동시에** 돌아오고, 그걸 모아 **사람이 판정**합니다.

중요한 건 **개수가 아닙니다.** 일이 정말 **독립적으로 나뉘는가**입니다.

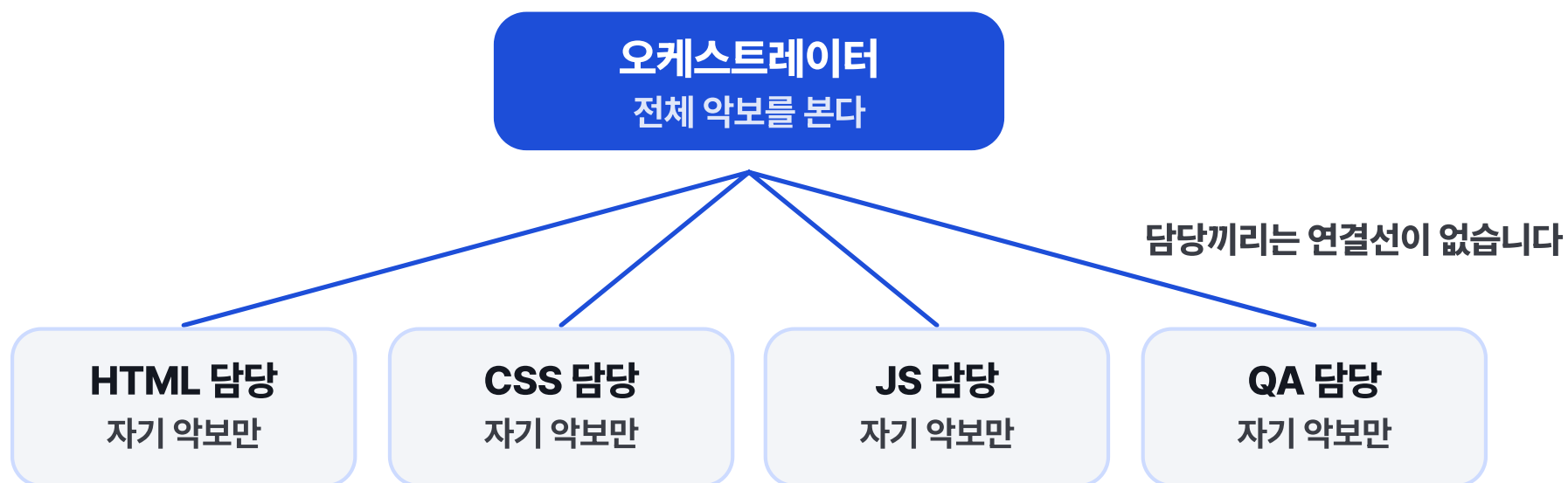


셋을 나란히 보면

	싱글	서브	멀티
통제권	에이전트 하나	메인이 유지	메인이 수집 · 판정
장면	한 요청 → 한 결과	메인 → 떼어낸 작업 → 반환	병렬 검토 → 비교 → 한 명이 구현
적합	작은 단일 수정	검토 · 읽기 같은 한 가지 일	서로 독립적인 검토 묶음

오케스트레이터

전체 순서와 역할표를 아는 지휘자 — 각 담당은 **자기 일만** 압니다. 옆 담당이 뭘 하는지 모릅니다.



오케스트라에서 연주자는 자기 악보만 보고, 전체 악보를 보는 건 지휘자뿐입니다.
오늘 그 자리는
여러분입니다.

나누면 무조건 좋을까요

아닙니다. 담당이 늘수록 **비용도 같이 늘어납니다.**

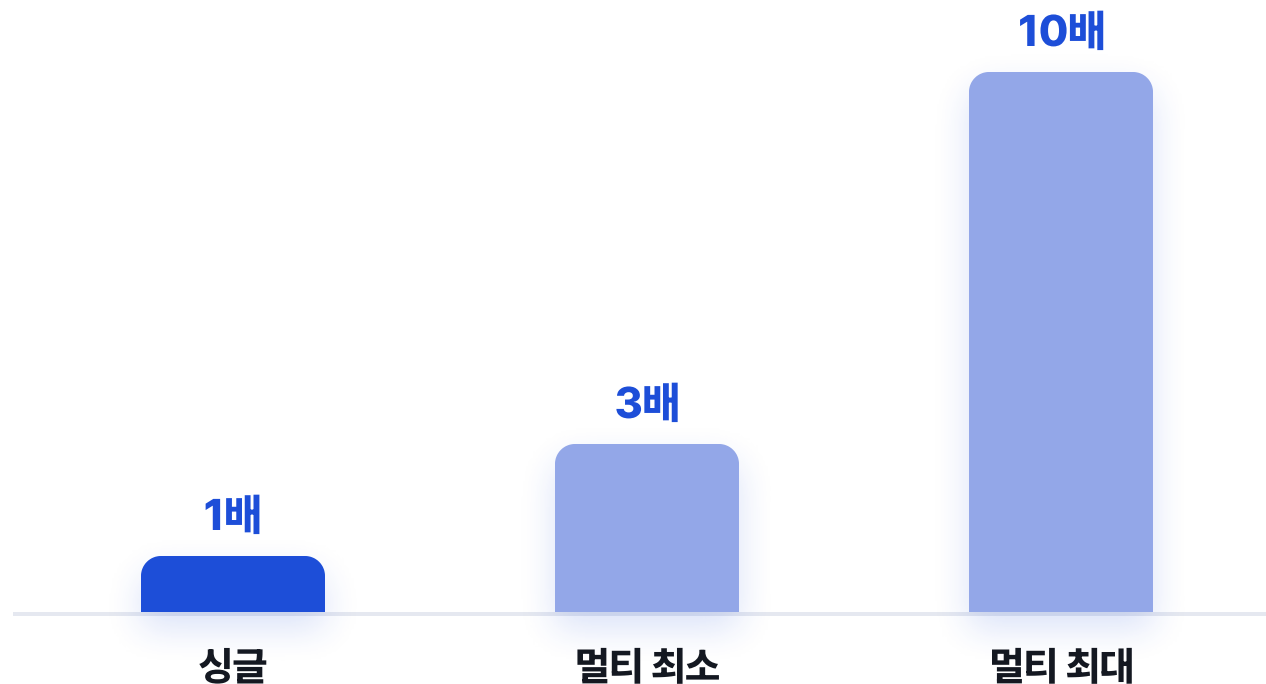
설명 중복

결과 충돌

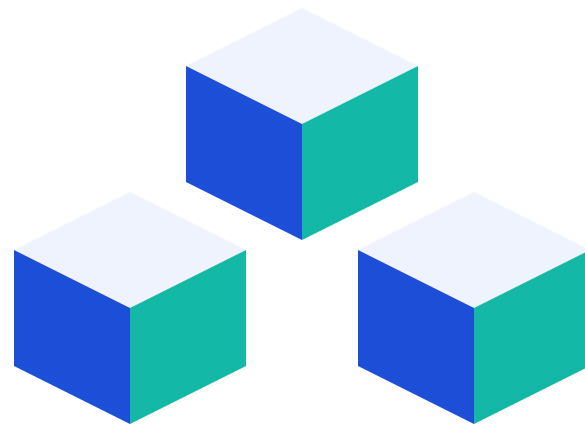
취합 비용

"에이전트가 많을수록 좋다"도, "병렬로 돌리면 무조건 빠르다"도 틀린 말입니다.

회의 참석자가 두 배가 되면 회의가 두 배 빨라지지 않는 것과 같습니다.



같은 일을 시켰을 때 쓰는 분량입니다. 나누는 순간부터 **이만큼을 더 씁니다.**



PART 3 / 6

담당 나누기와 팀 규칙

내 프로젝트를 넷으로 나눕니다.

PART 3은 이렇게 배웁니다

끝나면 누가 어느 파일을 맡는지가 파일에 남습니다.



우리 프로젝트는 조금 다릅니다

보통 팀은 이렇게 나눕니다

프론트엔드

~~백엔드~~

QA

~~데이터베이스~~

그런데 우리는 **프론트엔드만** 만들었습니다. 서버도
데이터베이스도 없습니다.

백엔드 담당을 불러도 할 일이 없습니다.

그래서 다른 기준으로 나눕니다

index.html

style.css

script.js

화면

기준은 **파일**입니다.

우리가 실제로 가진 것에 맞춰 나눕니다.

HTML · CSS · JS · QA 담당

사람 이름이 아니라 파일 이름이 담당 경계가 됩니다.

index.html



HTML 담당

구조를 봅니다

style.css



CSS 담당

스타일을 봅니다

script.js



JS 담당

동작을 봅니다

화면



QA 담당

파일이 아니라 화면을 검증합니다

경계가 파일이라 **검칠 수가 없습니다**. 이게 오늘 사고를 막아 줍니다.

실습 ④

내 프로젝트에 담당 배정하기

- 1 내 프로젝트를 HTML · CSS · JS · QA 네 담당으로 나눕니다.
- 2 각 담당이 어느 파일을 맡을지 한 줄씩 적습니다.
- 3 파일이 3개뿐이어도 괜찮습니다 — QA 담당은 파일 대신 화면을 맡습니다.

이렇게 말하세요 "내 프로젝트를 HTML(구조) · CSS(스타일) · JS(동작) · QA(검증) 네 담당으로 나눈다면 각각 어떤 파일을 맡으면 좋을지 제안해 줘. **파일은 아직 고치지 마.**"

완성 기준 — 파일마다 **담당자 하나씩** 정해짐

용어

단일 작성자

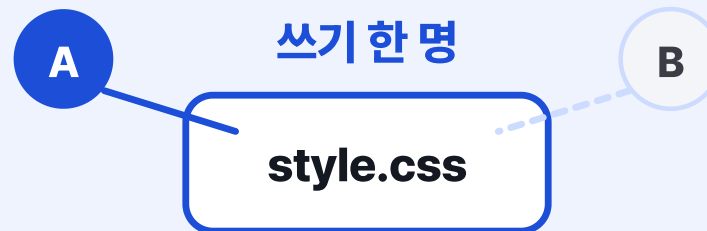
같은 수정 단계에서 **한 파일**을 쓰는 역할은 **한 명뿐**이라는 규칙

둘이 동시에 고치면



서로의 결과를 덮어씁니다.

쓰는 건 한 명입니다



읽는 건 여럿이 동시에 해도 됩니다.

QA 담당은 어떤 파일도 쓰지 않습니다.

팀 규칙을 다시 쓰는 이유

새로 만드는 게 아니라 지난주 것을 팀용으로 넓히는 겁니다.

2주차 · 혼자용

나 혼자 + 싱글 에이전트
목표 · 읽을 자료
권한 · 완료 조건
사후 확인

넓힌다
→

3주차 · 팀용

지난주 다섯 칸은 그대로 두고

누가 어느 파일을 맡나

남의 파일은 안 건드린다

새로 필요한 건 이 두 줄뿐입니다

오늘은 여러 담당이 같은 폴더를 봅니다. 전제가 달라졌습니다.

실습 ⑤

AGENTS.md에 팀 규칙 추가

1 기존 AGENTS.md에 세 가지를 더합니다 — 담당 파일 표.

2 남의 파일은 건드리지 않음, 그리고 끝내기 전에 확인할 것.

3 이미 있는 내용은 지우지 않고 덧붙이기만 합니다.

이렇게 말하세요 "지금부터 여러 담당이 이 프로젝트를 나눠서 볼 거야. AGENTS.md에 담당 파일 표, 남의 파일은 수정하지 않는다는 규칙, 완료 전에 확인할 항목을 추가해 줘. **기존 내용은 지우지 마.**"

완성 기준 — 세 항목이 파일에 실제로 적혀 있음

실습 ⑤ — 그대로 복사해 쓰세요

팀 구성 프롬프트

이렇게 말하세요

이 프로젝트에 **AI 개발팀**을 구성해 줘.
다음 네 개의 전문 서브에이전트를 만들어.

1. **HTML 담당** — 화면 구조 · index.html
2. **CSS 담당** — 스타일 · style.css
3. **JS 담당** — 동작 · script.js
4. **QA 담당** — 검증 · 화면 전체

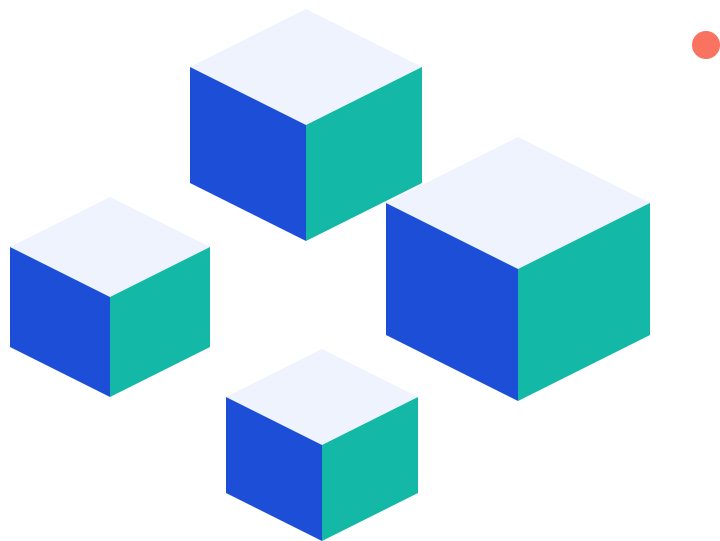
각 담당은 자신의 이름과 역할을 인식해야 한다. 역할을 질문받으면 "**나는 OO 담당이야.**"라고 답해야 한다.

각 담당은 **자기 파일에만** 집중하고, 메인은 요청에 따라 적절한 담당에게 작업을 위임한다.

이 구성이 이후 작업에도 유지되도록 필요한 **Codex 설정 파일과 AGENTS.md**를 생성해 줘.

초보자가 추가로 수정하지 않아도 **바로 사용할 수 있도록** 완성해 줘.

마지막에 **생성한 파일과 각 담당의 호출 방법**을 설명해 줘.



PART 4 / 6

병렬 검토 · 결과 판정 · 구현

팀을 돌리고, 엇갈리면 사람이 판정합니다.

PART 4는 이렇게 배웁니다

순서는 지난주와 똑같습니다 — 사람만 늘었습니다. 오늘 실습이 가장 많은 구간이라 손이 바빠집니다.



검토 요청에 넣을 것 셋

담당에게 검토를 맡길 때 반드시 세 가지를 함께 줍니다.

1

어느 파일만

볼 파일을 하나로 못 박습니다. 안 적으면 폴더 전체를 훑습니다.

2

무엇을 찾을지

찾을 대상을 정해 줍니다. 안 적으면 「좋아 보이는 것」을 아무거나 말합니다.

3

고치지 말 것

가장 자주 빠뜨리는 칸입니다. 이걸 빠뜨리면 검토만 시켰는데 파일이 바뀌어 있습니다.

셋 중 하나라도 비면 그 자리는 에이전트가 알아서 채웁니다.

실습 ⑥

담당별 병렬 검토 요청

1 담당 파일마다 따로 요청합니다 — 요청창을 담당 수만큼 나눠서 엽니다.

2 HTML · CSS · JS 담당 셋에게 같은 형식으로 이어서 보냅니다.

3 결과가 다 돌아올 때까지 아무것도 고치지 않습니다.

이렇게 말하세요 "지금 이 프로젝트에서 HTML 담당(화면 뼈대)만 맡아 줘. index.html만 읽고 개선하면 좋을 점 세 개를 근거와 함께 제안해 줘. **파일은 고치지 마.**" — CSS · JS 담당도 같은 형식으로 반복합니다.

완성 기준 — 담당별로 **서로 다른 제안**이 손에 있음

세 담당은 서로 다른 것을 봅니다

같은 화면 · 세 담당이 본 곳

제목 아래가 비어 있다

HTML

이 글자가 작아서 안 읽힌다

CSS

이 버튼이 눌러도 반응이 없다

JS

HTML 담당

화면에 빠진 것을 봅니다

CSS 담당

읽기 어려운 곳을 봅니다

JS 담당

눌렀을 때 안 되는 것을 봅니다

결과를 받으면 먼저 확인하세요 — 근거가 서로 다른가? 같은 말을 하면 나눈 의미가 없습니다.

강사 예시

결과 충돌

담당마다 다른 결론을 낸 상태 — 각자 다른 곳을 봤으니 **당연한 일**입니다.

HTML 담당

목록 맨 위에 큰 글씨로

CSS 담당

제목 옆에 작은 배지로

활동 기록 도구

오늘 5개

기록 목록

활동 기록 도구 **5**

기록 목록

기록 목록

둘 다 맞는 말입니다. 그래서 정해야 합니다.

다수결이 아니라 근거로 판정합니다

× "셋 중 둘이 그러니까"는 근거가 아닙니다. 같은 자료를 봤으면 같은 답이 나오는 게 당연합니다.

채택

근거를 확인했고 오늘 반영합니다.

부분 채택

일부만 맞아 그 부분만 씁니다.

기각

확인해 보니 근거가 틀렸습니다.

확인 불가

지금 화면으로는 판단할 수 없습니다.

실제 화면과 파일을 열어 보고 사람이 정합니다. 확인 불가도 결과입니다 — 모르는 것을 채택으로 밀어 넣지 않습니다.

실습 ⑦

결과 비교하고 판정하기

1

담당별 제안을 한 표에 모읍니다 — 제안 / 근거 / 판정.

2

판정 칸에 채택 · 부분 채택 · 기각 · 확인 불가를 직접 씁니다.

3

오늘 반영할 것은 한 건입니다. 나머지는 기록만 남깁니다.

제안	근거	판정
HTML 담당의 제안	어느 화면을 보고 그렇게 말했나	직접 씁니다
CSS 담당의 제안	어느 화면을 보고 그렇게 말했나	직접 씁니다
JS 담당의 제안	어느 화면을 보고 그렇게 말했나	직접 씁니다

완성 기준 — 오늘 고칠 제안이 **하나로 좁혀짐**

실습 ⑧

승인된 개선 구현

- 1 채택한 제안 하나만 담당 파일에 반영합니다.
- 2 그 파일의 담당 한 명에게만 시킵니다 — 단일 작성자입니다.
- 3 끝나면 대표 흐름을 브라우저에서 다시 실행합니다. 새 기능만 보고 넘어가지 않습니다.

이렇게 말하세요 "[승인된 개선]을 [담당 파일]에만 반영해 줘. 다른 파일과 기존 기능은 건드리지 말고, 끝나면 무엇을 바꿨는지 알려 줘."

완성 기준 — 기존 흐름과 새 흐름이 모두 작동

용어 들

독립 QA와 회귀

독립 QA — 구현에 참여하지 않은 담당이 하는 확인

회귀 — 잘 되던 기존 기능이 다시 깨지는 것

고친 사람은 "될 것"이라고 생각하면서 보기 때문에 같은 곳을 계속 놓칩니다.

그래서 QA 담당은 **직접 고치지 않고** 확인만 합니다. 자기 글 교정이 안 되는 이유와 같은 원리입니다.

고친 담당 — 익숙한 시선

새 기능

여기만 봅니다

기존 기능

= 회귀가 숨는 자리

QA 담당 — 처음 보는 시선

새 기능

둘 다 봅니다

기존 기능

실습 ⑨

독립 QA 실행

1 QA 담당에게 수정은 시키지 않고 확인만 시킵니다.

2 대표 흐름을 정상값 · 다른 값 · 빈 값 세 가지로 실행합니다.

3 각 항목을 통과 / 실패로 받아 적습니다.

이렇게 말하세요 "이번엔 **수정하지 말고 검증만** 해줘. 대표 흐름을 정상값 · 다른 값 · 빈 값으로 각각 실행해서 결과를 통과/실패로 알려줘."

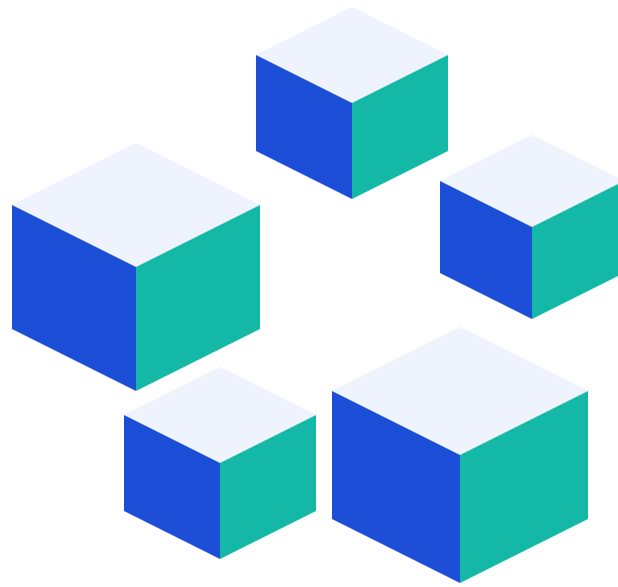
완성 기준 — 기존 흐름과 새 흐름이 **셋 다 통과**

화면 QA 6항목

담당이 "통과"라고 해도 화면을 직접 보는 것이 마지막 확인입니다.

확인할 것	통과	실패	해당 없음
기존 흐름	☐	☐	☐
새 흐름	☐	☐	☐
다른 값	☐	☐	☐
오류 상황	☐	☐	☐
좁은 화면	☐	☐	☐
글자 크기	☐	☐	☐

「해당 없음」은 그 기능이 없다는 뜻이지, **확인을 건너뛰었다는 뜻이 아닙니다.**



PART 5 / 6

플러그인 · MCP · GitHub 기록

도구를 붙이고, 기록을 남깁니다.

PART 5는 이렇게 배웁니다

GitHub 계정과 저장소 연결은 수업 전에 이미 끝나 있습니다. 오늘은 확인만 합니다.



지금까지는 내 폴더 안에서만

지금까지 에이전트는 내 폴더 밖을 몰랐습니다.



1 지금

바깥 자료를 읽지도, 남기지도 못했습니다.

2

도구를 붙이면

할 수 있는 일의 범위가 넓어집니다.

스마트폰에 앱을 깔면 새 일을 하게 되는 것과 같습니다. **본체는 그대로입니다.**

플러그인 · Skill · 커넥터 · MCP

넷은 경쟁하는 제품이 아니라 서로 다른 층입니다.

플러그인

여러 기능을 묶어 한 번에 설치하는 꾸러미 — 안에 아래 것들을 담고 있을 수 있습니다

Skill

반복 절차를 적어 둔 것

커넥터

특정 서비스에 로그인해 이어 주는 부분

MCP

AI와 바깥 도구를 잇는 공통 연결 규격

공구세트 안에 드라이버도 있고 콘센트 어댑터도 있는 것과 같습니다.

용어

MCP

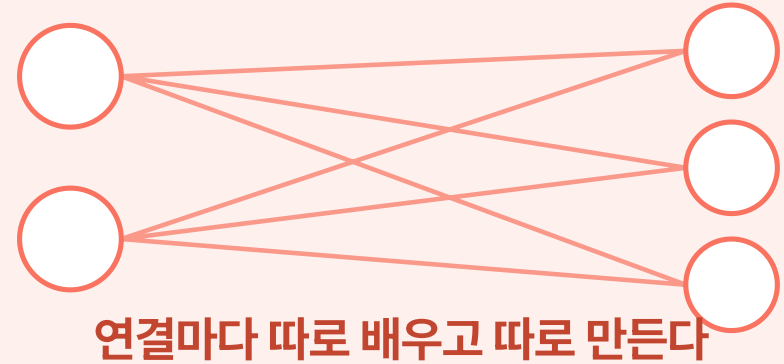
AI와 바깥 도구를 잇는 **공통 연결 규격**

도구마다 연결 방법이 다르면, 도구 수만큼 따로 배우고 따로 만들어야 합니다.

연결 방식을 **하나로 정해 두면** 한 번 배운 방법으로 여러 도구를 붙일 수 있습니다.

콘센트 규격이 같으면 어느 가전이든 꽂히는 것과 같습니다.

규격이 없으면

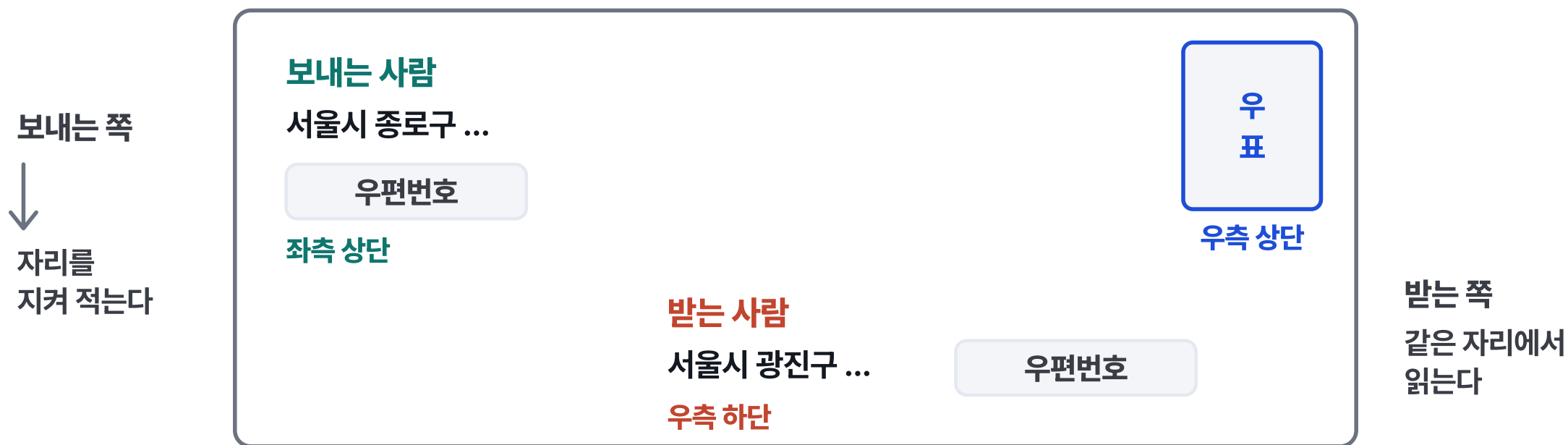


공통 규격을 두면



프로토콜

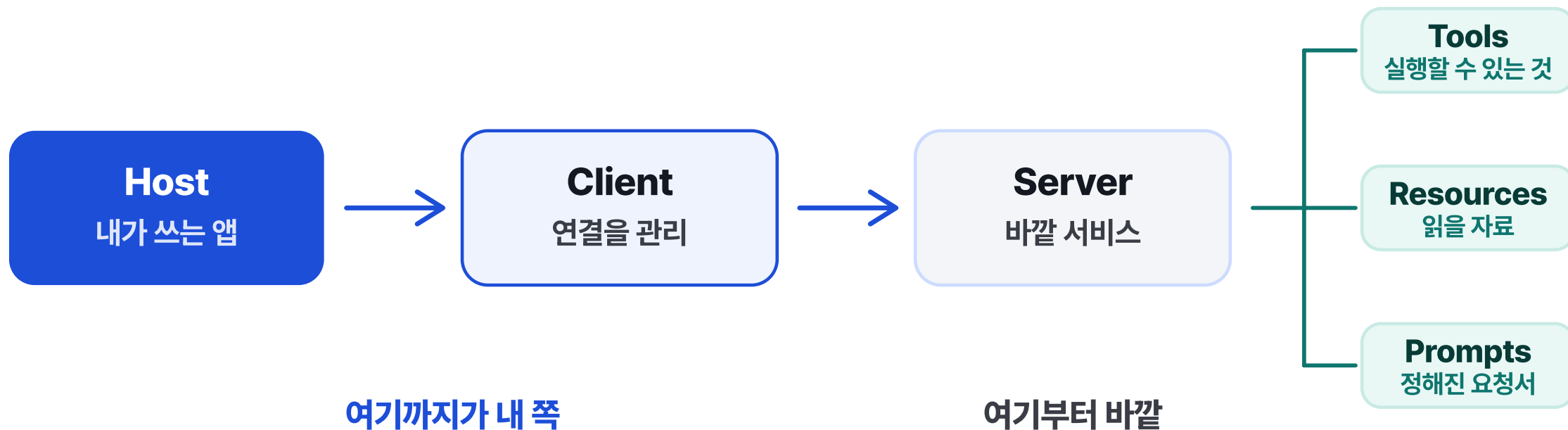
서로 다른 둘이 오해 없이 주고받기 위해 미리 맞춰 둔 약속 — MCP의 **P**가 이 P입니다.



봉투에 주소를 아무 데나 적으면 우체국이 못 읽습니다. **자리를 미리 정해 둔 것**이 프로토콜입니다.

MCP의 세 자리

이름을 외울 필요는 없습니다. 어디까지가 내 쪽이고 어디부터가 바깥인지만 보면 됩니다.



전기에 비유하면 Host는 가전, Client는 플러그, Server는 발전소입니다.

연결했다고 다 허락한 건 아닙니다

도구를 연결하면 AI는 손을 하나 더 갖게 됩니다. 그 손이 읽기만 할지, 바뀌도 되는지는 따로 정하는 일입니다.

연결됨

도구가 붙었습니다. 아직 아무것도 허락하지 않은 상태입니다.



승인 대기

무엇을 하는 도구인지 알고 나서 허락합니다.



승인 범위 초과

허락한 것보다 더 하려 하면 거기서 멈춰야 합니다.

연결과 승인은 다른 단계입니다. 택배 기사에게 문 앞까지 열어 준 것이지, 현관 비밀번호를 준 게 아닙니다.

용어 셋

GitHub — 저장소와 커밋

GitHub — 내 프로젝트를 인터넷에 보관하고 변경 이력을 남기는 서비스

저장소 — 내 프로젝트가 통째로 올라가는 자리

커밋 — 무엇을 왜 바꿨는지 적어 남기는 기록 한 건

사진첩과 같습니다 — 저장소가 앨범이고, 커밋은 날짜와 설명이 붙은 사진 한 장입니다.

저장소 = 앨범

첫 화면 만들기

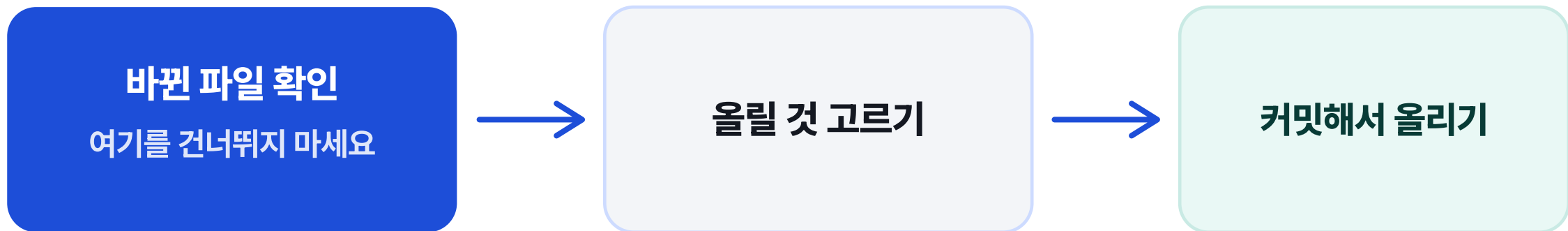
기록 추가 기능 넣기

삭제 후 화면 고치기

오늘 만들 커밋

커밋은 '무엇을 올릴지 고르는 것'부터입니다

바뀐 것을 다 올리지 않습니다. 고를 수 있습니다.



첫 칸을 건너뛰고 전부 담으면 올리면 안 되는 파일까지 딸려 갑니다

장바구니에 다 쓸어 담고 계산하는 것과 **하나씩 고르는 것**의 차이입니다.

실습 ⑩

저장소 연결과 변경 파일 확인

- 1 GitHub 저장소 연결이 살아 있는지 먼저 확인합니다.
- 2 오늘 바뀐 파일 목록을 아직 올리지 말고 눈으로 봅니다.
- 3 목록이 예상보다 길면 — 내가 모르는 변경이 섞였다는 뜻입니다.

이렇게 말하세요 "지금 이 프로젝트에서 어떤 파일이 바뀌었는지 목록으로 보여 줘. **아직 커밋하거나 올리지는 마.**"

완성 기준 — 올릴 후보 파일 목록을 **눈으로 확인함**

오늘 두 번째 안전 점검

올리기 전에 다시 확인합니다

PART 1에서 걸러 났어도 작업 중에 다시 들어옵니다. AI가 예시로 넣은 값일 수도 있습니다.

비밀번호

있으면 제외

API 키

있으면 제외

개인 연락처

있으면 제외

남의 자료

있으면 제외

내 컴퓨터 경로

있으면 제외

하나라도 있으면 **그 파일은 빼고** 올립니다. 한 번 올라간 커밋은 나중에 지워도 **흔적이 남습니다.**

실습 ⑪

커밋 메시지 한 문장 쓰기

X 수정함

X 완료

X ○○



✓ 빈 검색어일 때 안내 문구 표시
차이는 하나입니다 — 무엇을 왜 바꿨는지가 문장에 있는가.

1 PART 4에서 구현한 개선을 그런 한 문장으로 적습니다.

2 3개월 뒤의 나를 독자로 두고 씁니다.

완성 기준 — 3개월 뒤에 봐도 이해되는 문장 하나

실습 ⑫

커밋하고 GitHub에 올리기

- 1 확인한 파일만 골라 커밋하고, GitHub에 올립니다.
- 2 올리기 전에 무엇을 올리는지 한 번 더 보여 달라고 합니다.
- 3 확인한 다음에 진행합니다 — 이 순서를 지키면 사고가 안 납니다.

이렇게 말하세요 "지금 확인한 파일만 골라서, 아까 만든 문장으로 커밋하고 GitHub에 올려 줘. **올리기 전에 무엇을 올릴지 먼저 보여 주고, 내가 확인하면 진행해.**"

완성 기준 — 저장소에 **오늘 만든 커밋이 도착함**

"완료했습니다"보다 GitHub 화면

에이전트의 답변

"GitHub에 올렸습니다"

그건 보냈다는 말입니다. 도착했다는 말이 아닙니다.

GitHub 웹 화면 ✓

커밋 목록

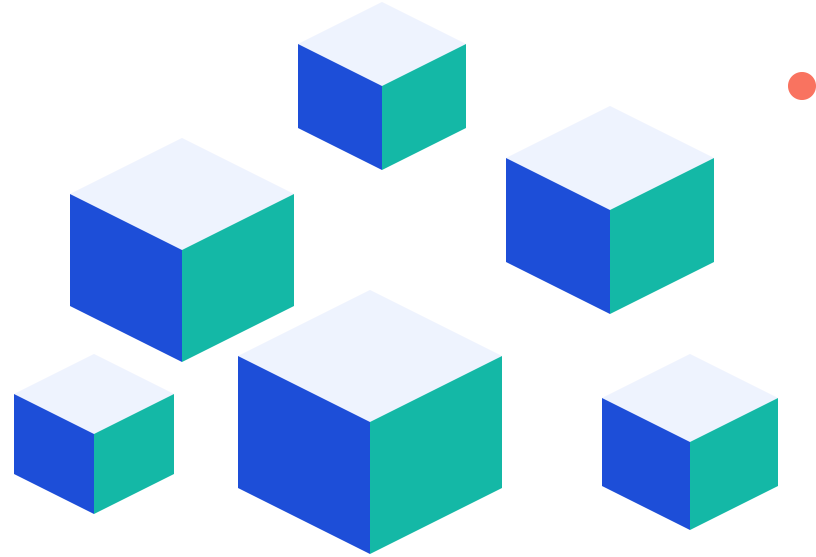
● 빈 검색어일 때 안내 문구 표시

방금

● 기록 추가 기능 넣기

최신 커밋이 실제로 보여야 끝난 겁니다.

완료 보고는 증거가 아닙니다. 증거는 화면입니다.



PART 6 / 6

공개와 공유 범위

세상에 내놓습니다.

PART 6은 이렇게 배웁니다

②와 ③은 다른 단계입니다. 만들었다고 공개된 게 아닙니다.



저장소에 있는 것과, 열리는 것은 다릅니다

저장소에 있는 것 — 파일

파일 목록

index.html

style.css

script.js

파일을 받아서 직접 열어 보라고 할 수는 없습니다.

사람들에게 필요한 것 — 열리는 URL

내-사이트.chatgpt.site

열리는 주소가 있어야 남이 씁니다.

레시피를 공유한 것과 **요리를 내놓은 것**의 차이입니다.

오늘 세 번째 안전 점검

공개하면 되돌리기 어렵습니다

공개하면 누구나 봅니다. 링크를 몰라도 검색으로 닿을 수 있습니다.

개인정보
화면에 남았나요?

남의 정보
화면에 남았나요?

미완성 안내문
화면에 남았나요?

테스트용 값
화면에 남았나요?

여기가 경계선
넘으면 회수 불가

지금 **마지막으로 되돌릴 수 있는 지점**입니다 — 오늘 세 번 한 점검 중 이번이 제일 중요합니다.

사이트 만들기

왼쪽 메뉴 사이트로 들어가, 오른쪽 위 만들기를 누릅니다.

Codex

새 채팅

풀 리퀘스트

사이트

예약

플러그인

사이트

만들기

사이트	공유 대상	
내 프로젝트 내-사이트.내 계정.chatgpt.site	🔒 나만	공유

주소는 <사이트 이름>.<내 계정>.chatgpt.site 모양으로 생깁니다. 만든 직후 공유 대상은 「나만」입니다.

실습 ⑬

공유 범위 바꾸고 URL 확인

1

만든 직후 공유 대상은 「나만」입니다 — 아직 아무도 못 봅니다.

2

사이트를 열고 공유 → 관리를 누릅니다.

3

범위를 인터넷 사용자 누구나로 바꾸고 게시를 누릅니다.

4

바뀐 주소를 다른 창이나 휴대폰에서 열어 봅니다.

공유

액세스 권한이 있는 사용자



인터넷 사용자 누구나

v

취소

게시

완성 기준 — 로그인하지 않은 상태에서도 내 프로젝트가 열림

공개가 안 되면

계정이나 기능 문제로 막히면 거기서 멈추지 않습니다.

공개 성공

공개 URL이 로그인 없이 열립니다.
그대로 다음 실습으로 갑니다.

폴백

로컬에서 실행한 화면을 캡처하거나
녹화해서 남깁니다.

미완료

어디까지 갔는지를 기록합니다. 안 된
것을 됐다고 적지 않습니다.

오늘의 목표는 "내 결과물이 밖에서 열린다"입니다. 방법이 막혀도 **경험은 지킵니다.**

공개된 화면은 내 컴퓨터와 다를 수 있습니다

파일 경로, 대문자·소문자, 캐시 때문입니다. 그래서 처음 방문한 사람처럼 네 가지를 봅니다.

첫 화면

주소를 열자마자 보이는 것이 의도한 화면인가

대표 흐름

가장 중요한 동작이 끝까지 되는가

좁은 화면

휴대폰 폭에서 글자와 버튼이 살아 있는가

새로고침

다시 불러와도 같은 화면이 나오는가

만든 사람 눈으로 보면 다 정상으로 보입니다. **일부러 낮설게 보세요.**

실습 ⑭ · 5분

공개 URL 자가 점검

- 1 내 공개 URL을 로그인하지 않은 창에서 엽니다.
- 2 첫 화면 · 대표 흐름 · 좁은 화면 · 새로고침을 차례로 확인합니다.
- 3 이상한 곳을 찾으면 한 가지만 적어 둡니다.

완성 기준 — 고칠 것 **한 건이 문장으로** 적힘

한 번에 하나만 고칩니다

여러 개를 찾아도 오늘은 **가장 중요한 하나만** 고칩니다.

한꺼번에 고치면 **뭐 때문에 나아졌는지** 알 수 없습니다.

3주 내내 지킨 원칙을 마지막까지 그대로 씁니다.



실습 ⑮ · 마지막

한 건 고치고 다시 공개

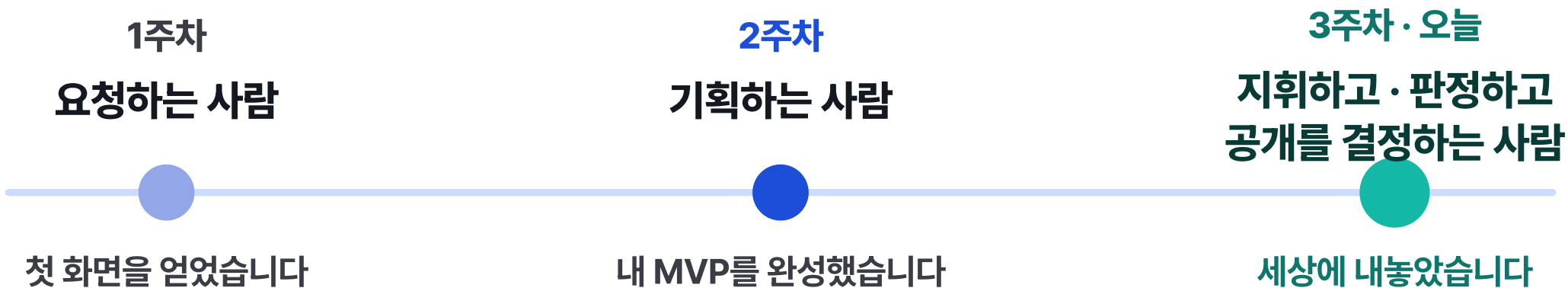
- 1 고른 한 건을 반영합니다. 다른 기능은 그대로 둡니다.
- 2 끝나면 다시 공개해 바뀐 화면이 URL에 반영됐는지 확인합니다.
- 3 옛 화면이 보이면 고장이 아니라 캐시입니다 — 새로고침하면 됩니다.

이렇게 말하세요 "공개 URL에서 확인한 이 문제만 고쳐 줘: [문제]. 다른 기능은 그대로 두고, 반영되면 다시 공개해 줘."

완성 기준 — 공개 URL에서 **변경이 실제로 보임**

3주 회고

AI가 여럿이어도 채택은 사람이 하고, 완료는 증거로 결정합니다.



이 문장 하나만 가져가시면 **오늘 수업은 성공**입니다.

3주 회고

오늘 한 문장으로 남기기

AI가 여럿이어도 **채택은 사람이 하고, 완료는 증거로 결정합니다.**

1주차 · 요청하는 사람



2주차 · 기획하는 사람



3주차 · 지휘하고 판정하고 공개하는 사람

나눴습니다

파일을 경계로 담당 넷을 두었습니다

판정했습니다

다수결이 아니라 근거로 골랐습니다

기록했습니다

무엇을 왜 바꿨는지 커밋에 남겼습니다

공개했습니다

로그인 없이 열리는 것을 확인했습니다

검증된 것만 완료로 봅니다.

3주간 고생하셨습니다 · THANK YOU